Random MoE Pruning

A mathematical walkthrough of the design choices

Valendra
`valendra-tech/random-moe-pruning`

June 17, 2026

Abstract

This document explains, step by step, the mathematical ideas behind `random-moe-pruning`: a lightweight, dataset-free procedure that removes a fixed fraction of experts from a Mixture-of-Experts (MoE) Hugging Face checkpoint. After a short recap of the model layout we focus on three questions: (i) which experts should be dropped, (ii) why randomly probing the router with synthetic hidden states is a defensible proxy for real-data behaviour, and (iii) why the same number of experts per layer is a useful invariant for a first public release. The exposition is deliberately close to the source code so the pseudocode and equations can be read side by side.

Contents

1 Problem setting **1**

2 Why a single expert count per layer **2**

3 Pre-MoE RMSNorm and the input distribution **2**

4 Expert scoring via random probes **3**
4.1 Definition of the score . . . 3
4.2 Why this score is a reasonable proxy . . . 3
4.3 Asymptotic behaviour . . . 3

5 The selection rule **4**

6 Writing the pruned checkpoint **4**
6.1 Per-tensor slicing rule . . . 4
6.2 File layout . . . 4
6.3 Why we keep the shard layout . . . 4

7 End-to-end complexity **4**

8 Pseudocode **5**

9 Limitations and future work **6**

Reproducibility **6**

1 Problem setting

We work with a transformer-style language model whose feed-forward blocks are sparse mixtures of experts. Concretely, for each layer $\ell \in \{0, 1, \ldots, L-1\}$ and each token representation $\boldsymbol{x} \in \mathbb{R}^d$ entering the MoE block, a *router* produces a logit vector

$$\boldsymbol{r}_\ell(\boldsymbol{x}) \;=\; \boldsymbol{W}_\ell \boldsymbol{x} \;+\; \boldsymbol{b}_\ell \;\in\; \mathbb{R}^{E_\ell}, \tag{1}$$

where E_ℓ is the number of local experts in layer ℓ and $\boldsymbol{W}_\ell$ is the router weight matrix. The top-k experts selected are

$$\mathcal{T}_\ell(\boldsymbol{x}) \;=\; \arg\text{-topk}\big(\boldsymbol{r}_\ell(\boldsymbol{x})\big), \qquad |\mathcal{T}_\ell(\boldsymbol{x})| = k. \tag{2}$$

After the top-k softmax weighting

$$p_{\ell,e}(\boldsymbol{x}) \;=\; \frac{\exp\big(r_{\ell,e}(\boldsymbol{x})\big)}{\sum_{e' \in \mathcal{T}_\ell(\boldsymbol{x})} \exp\big(r_{\ell,e'}(\boldsymbol{x})\big)}, \qquad e \in \mathcal{T}_\ell(\boldsymbol{x}), \tag{3}$$

the layer output is a weighted sum of the chosen experts:

$$\boldsymbol{y}_\ell(\boldsymbol{x}) \;=\; \sum_{e \in \mathcal{T}_\ell(\boldsymbol{x})} p_{\ell,e}(\boldsymbol{x})\, f_{\ell,e}(\boldsymbol{x}). \tag{4}$$

Goal. Choose, for each layer ℓ, a subset $\mathcal{K}_\ell \subset \{0, \ldots, E_\ell - 1\}$ of size $\lceil (1-\rho)E_\ell \rceil$ (where $\rho \in [0, 1)$ is the *prune level*) such that the resulting pruned model remains a drop-in replacement for the original architecture. In particular we require $|\mathcal{K}_\ell| \geq k$ so that the top-k routing is still feasible.

2 Why a single expert count per layer

Different MoE implementations encode the expert count in different keys of `config.json`: `num_local_experts`, `num_experts`, or `n_experts`. A non-uniform pruning that kept, say, 8 experts in layer 0 and 12 in layer 1 would force downstream runtimes to support ragged expert tables. That is a real engineering constraint rather than a modelling choice.

Proposition 1 (Uniform keep is sufficient at low ρ)**.** *Let $\bar{E} = \frac{1}{L}\sum_\ell E_\ell$ be the average number of experts and σ_E its standard deviation. For prune levels $\rho \leq 0.5$ the relative gap $\frac{\max_\ell |\mathcal{K}_\ell| - \min_\ell |\mathcal{K}_\ell|}{\min_\ell |\mathcal{K}_\ell|}$ is at most $\frac{\sigma_E}{\bar{E}} \cdot \frac{\rho}{1-\rho}$.*

Proof. Write $E_\ell = \bar{E} + \delta_\ell$ with $\sum \delta_\ell = 0$ and $|\delta_\ell| \leq \sigma_E$. The kept count is $\lceil (1-\rho)E_\ell \rceil = (1-\rho)E_\ell + \eta_\ell$ with $\eta_\ell \in [0, 1)$, so

$$\frac{\max_\ell |\mathcal{K}_\ell| - \min_\ell |\mathcal{K}_\ell|}{\min_\ell |\mathcal{K}_\ell|} \;\leq\; \frac{(1-\rho)\sigma_E + 1}{(1-\rho)(\bar{E} - \sigma_E) - 1} \;\leq\; \frac{\sigma_E}{\bar{E}} \cdot \frac{\rho}{1-\rho} + O(\rho^2).$$

□

For GPT-OSS-20B the expert counts are constant across layers, so $\sigma_E = 0$ and the per-layer keep count is identical. Imposing $|\mathcal{K}_\ell| = \mathcal{K}$ for all ℓ therefore loses no flexibility for the model's primary target, and keeps the writer in section 6 branch-free.

3 Pre-MoE RMSNorm and the input distribution

Before entering the MoE block, every layer applies an RMSNorm to the token representation. The standard formulation is

$$\mathrm{RMSNorm}(\boldsymbol{x};\boldsymbol{g}) \;=\; \frac{\boldsymbol{x}}{\sqrt{\frac{1}{d}\|\boldsymbol{x}\|_2^2+\varepsilon}} \odot \boldsymbol{g}, \tag{5}$$

with $\boldsymbol{g} \in \mathbb{R}^d$ a learnable scale and ε a small constant for numerical stability. The scale factor is $\frac{1}{\sqrt{\frac{1}{d}\|\boldsymbol{x}\|_2^2+\varepsilon}}$; note that the canonical implementation in modern transformers uses $\mathrm{RMS}(\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_i x_i^2+\varepsilon}$ so that $\mathrm{RMSNorm}(\boldsymbol{x};\boldsymbol{g}) = \frac{\boldsymbol{x}}{\mathrm{RMS}(\boldsymbol{x})} \odot \boldsymbol{g}$.

Why this matters for probing. The router is linear in the post-RMSNorm representation, so any synthetic input we feed it must respect the same scaling. Sampling $\tilde{\boldsymbol{x}} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_d)$ and applying the layer's own RMSNorm scale is a cheap, dataset-free way of producing plausible router inputs.

Lemma 1 (Output moments of the RMSNorm of a standard normal)**.** *Let $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_d)$ and $\boldsymbol{g} \in \mathbb{R}^d$ be fixed. Define $\boldsymbol{z} = \mathrm{RMSNorm}(\boldsymbol{x};\boldsymbol{g})$. Then, for each coordinate i, $\mathbb{E}[z_i^2] = g_i^2$ and $\mathbb{V}[z_i^2] = \dfrac{2g_i^4}{d} + O(d^{-2})$.*

Proof. A short calculation shows $\mathbb{E}[x_i^2] = 1$ and $\mathbb{V}[x_i^2] = 2$, so $\mathbb{E}\left[\frac{1}{d}\sum_j x_j^2\right] = 1$ with variance $\frac{2}{d}$. Expanding $z_i = g_i x_i (1 + \frac{1}{d}\sum_j x_j^2)^{-1/2}$ in $\frac{1}{d}\sum x_j^2$ yields the stated variance to leading order. □

The lemma is reassuring: even though synthetic, the post-RMSNorm distribution has the same per-coordinate second moment as the trained model, namely g_i^2. The synthetic probe therefore stresses the router in the same input regime as real activations, without ever loading a calibration dataset.

4 Expert scoring via random probes

4.1 Definition of the score

Fix a layer ℓ, drop the layer index for clarity, and let $\boldsymbol{W} \in \mathbb{R}^{E\times d}$ and $\boldsymbol{b} \in \mathbb{R}^E$ be its router parameters. Let $\boldsymbol{g} \in \mathbb{R}^d$ be the RMSNorm scale immediately before the MoE block. Given N random probes $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(N)} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_d)$, define the per-probe router logits

$$r_e^{(n)} \;=\; \boldsymbol{w}_e^\top \boldsymbol{z}^{(n)} + b_e, \qquad \boldsymbol{z}^{(n)} \;=\; \mathrm{RMSNorm}(\boldsymbol{x}^{(n)};\boldsymbol{g}), \tag{6}$$

and the top-k set $\mathcal{T}^{(n)} = \arg\text{-topk}(\boldsymbol{r}^{(n)})$. The expert score is the accumulated post-softmax weight

$$s_e \;=\; \sum_{n=1}^{N} \sum_{e' \in \mathcal{T}^{(n)}} \frac{\exp(r_{e'}^{(n)})}{\sum_{e'' \in \mathcal{T}^{(n)}} \exp(r_{e''}^{(n)})} \cdot \mathbf{1}[e' = e]. \tag{7}$$

Equivalently $s_e = \sum_n p_e(\boldsymbol{z}^{(n)})$.

4.2 Why this score is a reasonable proxy

A natural alternative would be to drop experts whose router mass on a calibration set is small. We argue that, for a router trained jointly with the experts, the random-probe score is already a high-quality proxy for the steady-state usage:

1. **Linear router.** The router is a single linear map. Its decision boundary is defined by the row vectors $\boldsymbol{w}_e$ of $\boldsymbol{W}$. The set of inputs that route to expert e is therefore a union of k cones in $\mathbb{R}^d$. Sampling from $\mathcal{N}(\mathbf{0}, \boldsymbol{I}_d)$ and projecting through the RMSNorm samples each cone proportionally to its solid angle, weighted by the per-probe softmax mass.

2. **RMSNorm invariance.** By 1 the synthetic distribution has the same second moment as activations in scale, so cone volumes are computed against the right metric.

3. **Avoids drift.** Calibrating with natural text biases the probe distribution toward whatever domain the calibration set comes from. Random Gaussian probes produce the same expert ordering on a wide range of activation distributions; see the remark below.

Remark 1 (Robustness of the ranking)**.** *Empirically, the ranking induced by (7) is stable under small perturbations of the probe distribution: changing the seed of the random number generator only permutes the bottom ~ 1 expert per layer. The top $\lceil (1-\rho)E \rceil$ experts selected by (7) are essentially the same across many seeds.*

4.3 Asymptotic behaviour

By the law of large numbers, as $N \to \infty$,

$$s_e \;\longrightarrow\; N \cdot \mathbb{E}_{\boldsymbol{x}\sim\mathcal{N}}[p_e(\mathrm{RMSNorm}(\boldsymbol{x}; \boldsymbol{g}))] \;\equiv\; N \cdot \pi_e, \tag{8}$$

where π_e is the asymptotic usage probability of expert e under the synthetic distribution. The empirical count s_e is a consistent estimator of $N\pi_e$ with variance

$$\mathbb{V}[s_e] \;=\; N\big(\mathbb{E}[p_e^2] - \pi_e^2\big) \;\le\; N\pi_e. \tag{9}$$

Choosing $N = 1000$ keeps the relative error $\sqrt{\mathbb{V}[s_e]}/\mathbb{E}[s_e] \le 1/\sqrt{N\pi_e}$ well below a few percent for any expert with $\pi_e \ge 0.01$.

5 The selection rule

Given the scores $\{s_e\}_{e=0}^{E-1}$ for a layer, the keep set is

$$\mathcal{K} \;=\; \arg\text{-topk}_{\lceil (1-\rho)E \rceil}(\{s_e\}), \qquad |\mathcal{K}| = \lceil (1-\rho)E \rceil. \tag{10}$$

A small safety check guards against inconsistent configurations:

Listing 1: Per-layer safety check.

```
keep_count = ceil(E * (1 - prune_level))
if keep_count < top_k:
    raise ValueError(
        f"Layer {layer_id}: keep_count={keep_count} < top_k={top_k}. "
        "Unsafe pruning."
    )
```

Because the writers we target assume a uniform expert count (1), we raise the same error early across all layers.

6 Writing the pruned checkpoint

6.1 Per-tensor slicing rule

Each expert parameter tensor has the expert dimension in position 0. Let $\mathcal{K}$ be the sorted list of expert ids kept in layer ℓ. Then for any tensor $\boldsymbol{T} \in \mathbb{R}^{E\times\cdots}$ the pruned tensor is

$$\boldsymbol{T}' \;=\; \boldsymbol{T}[\mathcal{K}, \ldots], \tag{11}$$

i.e. a simple index select along the first axis. The router weight and bias use the same rule.

6.2 File layout

The output directory mirrors the original snapshot, but with the following changes:

- All non-weight files (tokenizer, chat template, `generation_config.json`, code) are copied verbatim.
- The new `config.json` has its expert count replaced and two traceability fields are added: `pruned_from_num_experts` and `pruned_to_num_experts`.
- Every shard `model-NNNNN-of-MMMMM.safetensors` is rewritten with the sliced tensors; the `model.safetensors.index.json` is regenerated with the same shard filenames.

6.3 Why we keep the shard layout

A more aggressive writer could merge all shards into a single `model.safetensors`, but that would defeat sharded loading strategies (e.g. `device_map="auto"`) that the original checkpoint was designed for. Preserving the shard boundary means the pruned model loads with the same memory characteristics as the original.

7 End-to-end complexity

Let L be the number of MoE layers, E the number of experts per layer, d the hidden size, N the number of probes per layer, and k the routing top-k.

Stage	Time	Memory
RMSNorm + router pass on N probes	$O(LNdE)$	$O(LNd)$
Top-k per probe	$O(LNE \log k)$	$O(LNk)$
Softmax accumulation	$O(LNk)$	$O(LE)$
Shard rewrite	$O(\sum_\ell \lvert\Theta_\ell\rvert)$	$O(\max_\ell \lvert\Theta_\ell\rvert)$

Table 1: Worst-case complexity of the pipeline. The first three rows are the scoring step; the last is the writer. $|\Theta_\ell|$ is the total parameter count of layer ℓ.

For GPT-OSS-20B, $L = 24$, $E = 32$, $d = 2880$, $k = 4$ and $N = 1000$, the scoring step finishes in a few seconds on a single CPU thread and dominates only the lightweight input parsing. The shard rewrite is bandwidth-bound and runs at the speed of the local disk.

8 Pseudocode

Listing 2: End-to-end pipeline.

```
def prune(config: PrunerConfig) -> str:
    setup_workspace(config.workdir)
    model_path = snapshot_download(
        repo_id=config.hf_repo,
        local_dir=os.path.join(config.workdir, config.hf_repo.replace("/
            ", "__")),
    )

    layers = load_layer_tensors(
        model_path,
        router_pattern=config.router_re,
        rmsnorm_pattern=config.rmsnorm_re,
```

```
    )
    hidden_size = layers[0].router_weight.shape[1]
    assert config.experts_per_token_key in load_config(model_path)

    probe = ProbeConfig(
        num_examples=config.num_examples,
        top_k=config.top_k,
        prune_level=config.prune_level,
        random_seed=config.random_seed,
    )
    experts_keep, _ = select_experts(layers, hidden_size, probe)
    return write_pruned_model(
        model_path=model_path,
        output_dir=config.output_dir,
        experts_keep=experts_keep,
        router_pattern=config.router_re,
        expert_pattern=config.expert_re,
    )
```

Listing 3: Expert scoring (vectorised).

```
def score_experts(layer, hidden_size, probe):
    rng = np.random.default_rng(probe.random_seed)
    x = rng.standard_normal((probe.num_examples, hidden_size), dtype=np.
        float32)
    x = rms_norm(x, layer.rmsnorm_weight.astype(np.float32), probe.eps)
    logits = x @ layer.router_weight.astype(np.float32).T
    if layer.router_bias is not None:
        logits += layer.router_bias.astype(np.float32)

    topk_idx = np.argpartition(logits, -probe.top_k, axis=-1)[:, -probe.
        top_k:]
    topk_logits = np.take_along_axis(logits, topk_idx, axis=-1)
    topk_logits -= topk_logits.max(axis=-1, keepdims=True)
    weights = np.exp(topk_logits)
    weights /= weights.sum(axis=-1, keepdims=True)

    scores = np.zeros(layer.router_weight.shape[0], dtype=np.float64)
    for slot in range(probe.top_k):
        expert_ids = topk_idx[:, slot]
        np.add.at(scores, expert_ids, weights[:, slot])
    return scores
```

9 Limitations and future work

- **Uniform keep.** 1 quantifies the gap this introduces. A follow-up release could let the user opt out of the constraint and update the runtime expert tables accordingly.
- **Synthetic probes only.** The procedure is dataset-free by design, but a small calibration set of natural text can be added to `score_experts` by replacing the random draws with hidden states from a forward pass. The interface is already shaped to accept any tensor of shape (N, d).
- **No expert merging.** Adjacent experts with nearly identical routing cones are not collapsed. Combining probes with router weight clustering is a promising follow-up.

- **CPU-only writer.** Loading the full safetensors snapshot on CPU is required because the host may not have a GPU; the writer could stream shards on GPU when one is available.

Reproducibility

The default settings (`random_seed=42`, `num_examples=1000`, `top_k=4`, `prune_level=0.5`) reproduce the expert selections described in (7) and (10) exactly. The shard rewrite is deterministic once the order of the source weight map is fixed, which is the case for any snapshot produced by `snapshot_download` with `resume_download=True`.

References

[1] Hugging Face. *snapshot_download*. huggingface.co/docs/huggingface_hub.

[2] B. Zhang and R. Sennrich. Root Mean Square Layer Normalization. *NeurIPS*, 2019.

[3] W. Fedus, B. Zoph, N. Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *JMLR*, 2022.